

# BÀI TẬP VỀ NHÀ – Ôn tập Bài 7 → 12

**Phạm vi ôn tập:** Giai đoạn 3 - Kiến trúc Component

**Nguyên tắc quan trọng:** KHÔNG động vào project Smart Recipe Box. Toàn bộ bài tập làm trên project riêng bai-tap-tra-sua, để dự án chính giữ nguyên trạng thái sẵn sàng cho Giai đoạn 4.

## Chuẩn bị

**Nếu bạn CHƯA có project bai-tap-tra-sua:**

```
cd <thư mục chứa project của bạn>
ng new bai-tap-tra-sua --style=css --ssr=false --routing=false --skip-tests
cd bai-tap-tra-sua && ng serve --port 4300
```

Chọn style=css (CSS thuần) chứ đừng chọn Tailwind, vì Nhiệm vụ C10 là bài tập tự viết CSS bằng tay. Nếu bạn đã tạo project với Tailwind rồi thì vẫn làm được, chỉ cần khi tới C10 thì viết CSS thuần vào file .css như bình thường.

**Nếu bạn ĐÃ làm bài tập 1-6:**

Bỏ qua phần code khởi đầu bên dưới, dùng luôn project cũ của bạn.

**Code khởi đầu (trạng thái tương đương "đã xong Bài 6"):**

Tạo 2 file trong src/app/ và dán nội dung vào. Sau đó thay nội dung app.ts và app.html.

- src/app/models.ts
- src/app/mock-drinks.ts
- src/app/app.ts
- src/app/app.html

Chạy ng serve port 4300, thấy trang hiện ra là bạn đã sẵn sàng bắt đầu.

## PHẦN A – Lý thuyết

Trả lời ngắn gọn bằng lời của bạn.

1. Phân biệt {{ }} và [ ]. Cả hai đều đưa dữ liệu ra màn hình, vậy tại sao cần tới hai cái?

**{{ }} (Interpolation):** Dùng để chèn chuỗi/chữ trực tiếp vào giữa hai thẻ HTML (ví dụ: <p>{{ name() }}</p>).

**[ ] (Property Binding):** Dùng để gán giá trị của biến vào thuộc tính (property) của thẻ HTML (ví dụ: [src]="imageUrl()", [disabled]="isLocked()").

**Vì sao cần hai cái:** Interpolation chỉ làm việc với dạng chuỗi hiển thị. Còn Property Binding có thể truyền được các kiểu dữ liệu phức tạp (boolean, number, object...) trực tiếp vào thuộc tính thẻ hoặc cổng input() của component con.

2. Vì sao <button disabled="{{ isLocked() }}"> là sai, còn <button [disabled]="isLocked()"> là đúng?

**Khi dùng `disabled="{{ isLocked() }}"`:** Angular sẽ biến giá trị thành một chuỗi (string) "true" hoặc "false". Trong HTML, chỉ cần thẻ button có thuộc tính disabled (dù giá trị chuỗi là gì), trình duyệt đều sẽ khóa nút lại!

**Khi dùng `[disabled]="isLocked()"`:** Angular sẽ gán thẳng giá trị kiểu boolean (true hoặc false) vào thuộc tính DOM disabled, giúp nút khóa/mở chính xác theo trạng thái.

3. Trong ví dụ `<img [src]="drink().imageUrl" width="300">`, tại sao src có ngoặc vuông mà width thì không?

**[src] có ngoặc vuông:** Vì giá trị bên trong ngoặc kép là một biểu thức TypeScript cần Angular tính toán để lấy đường dẫn động từ signal drink().

**width="300" không có ngoặc vuông:** Vì đây là một chuỗi hằng số cố định (300px), không thay đổi theo logic code nên truyền trực tiếp dạng HTML thuần.

4. Kể ra ba bước để dùng một component con bên trong component cha. Thiếu bước nào thì chuyện gì xảy ra?

- **3 bước chuẩn:**

1. import class của component con vào file .ts của component cha.
2. Thêm class con đó vào mảng imports: [...] trong decorator @Component của cha.
3. Gọi tên selector của component con trong file .html của cha (ví dụ: `<app-drink-detail />`).

- **Thiếu bước nào:** Nếu thiếu bước 1 hoặc 2, Angular sẽ không nhận diện được thẻ HTML tùy chỉnh và báo lỗi báo Unknown element hoặc ứng dụng bị đổ vỡ khi biên dịch.

4. Vì sao component App nằm thẳng trong src/app/, còn các component khác lại có thư mục riêng?

App (thường là AppComponent) là Component gốc (Root Component) quản lý toàn bộ ứng dụng, nằm ở thư mục gốc để làm mốc khởi tạo.

Các component khác tạo sau bằng Angular CLI (ng generate component name) sẽ tự động được gom vào một thư mục riêng chứa đủ 4 file (.ts, .html, .css, .spec.ts) để giữ cấu trúc thư mục gọn gàng, mô-đun hóa dễ quản lý.

5. Khi code chuyển từ src/app/app.ts xuống src/app/drink-list/drink-list.ts, tại sao import { DrinkModel } from './models' phải đổi thành '../models'?

'./' có nghĩa là tìm file ở **cùng cấp thư mục**. File app.ts nằm cùng cấp với models.ts nên dùng './models'.

Khi xuống drink-list/drink-list.ts, file này nằm trong thư mục con drink-list. Muốn tìm file models.ts ở ngoài src/app/, ta phải dùng '../models' để **lùi ra 1 cấp thư mục**.

6. Vì sao một input() không được đặt protected, trong khi signal thường thì nên có?

**signal() thường:** Là dữ liệu nội bộ của component đó, nên đặt protected để tránh việc các component khác truy cập linh tinh ngoài phạm vi cho phép.

**input():** Là cổng nhận dữ liệu do **Component Cha** truyền từ ngoài vào. Nếu để protected, Angular sẽ không thể gán dữ liệu từ Component Cha vào được, do đó input() bắt buộc phải dùng readonly và bỏ protected.

7. Phân biệt [drink]="selectedDrink()" và [drink]="selectedDrink". Cái nào đúng và tại sao?

**[drink]="selectedDrink()" là ĐÚNG:** Vì selectedDrink là một Signal. Ta cần cặp dấu ngoặc () để gọi hàm lấy giá trị thực sự (Object DrinkModel) bên trong truyền xuống cho con.

**[drink]="selectedDrink" là SAI:** Nếu không có (), bạn đang truyền cả cái hàm Signal xuống chứ không phải dữ liệu trà sữa

8. CSS viết trong drink-detail.css có ảnh hưởng tới thẻ <h2> ở component khác không? Giải thích cơ chế.\

**Không ảnh hưởng.**

**Cơ chế:** Angular sử dụng cơ chế **View Encapsulation** (mặc định là Emulated). Khi biên dịch, Angular sẽ tự động gán một attribute duy nhất (ví dụ: \_ngcontent-ng-c123) vào các thẻ HTML trong DrinkDetail và thêm attribute đó vào CSS Selector. Do đó, CSS của component nào thì chỉ có tác dụng đúng phạm vi component đó, không sợ rò rỉ ra bên ngoài

9. @if và CSS display: none khác nhau ở điểm nào? Nêu một tình huống mà sự khác biệt đó thật sự quan trọng.

- **Khác nhau:**

1. @if: **Thêm hoặc xóa hẳn** phần tử ra khỏi cây DOM HTML.

2. display: none: Phần tử **vẫn tồn tại trên cây DOM**, chỉ bị ẩn đi về mặt thị giác.

- **Tình huống quan trọng:** Khi phần tử đó chứa các đoạn code nặng, bộ lắng nghe sự kiện, hoặc dữ liệu cần bảo mật (như form thanh toán chứa thông tin thẻ). Dùng @if sẽ giải phóng bộ nhớ và không cho người dùng lén bật F12 (DevTools) để xem hay chỉnh sửa thông tin bị ẩn.

10. track trong @for dùng để làm gì? Vì sao track \$index thường là lựa chọn tệ?

- Tác dụng: track giúp Angular định danh duy nhất từng phần tử trong danh sách để khi danh sách bị thêm/xóa/sắp xếp lại, Angular chỉ vẽ lại đúng phần tử thay đổi thay vì vẽ lại toàn bộ danh sách (tăng hiệu năng).

- Vì sao track \$index tệ: \$index là vị trí thứ tự (0, 1, 2...). Khi bạn xóa phần tử đầu tiên, phần tử thứ hai sẽ bị đẩy lên vị trí 0. Angular tưởng rằng phần tử đó chính là phần tử cũ và có thể gây ra lỗi hiển thị sai dữ liệu hoặc sai hiệu ứng hoạt họa (animation). Do đó nên dùng một thuộc tính duy nhất như drink.id

11. Có cần import gì để dùng @for và @if không? So sánh với ngFor/ngIf ngày xưa.

**Không cần import gì cả:** Các cú pháp `@for`, `@if`, `@else`, `@switch` là cú pháp tính năng mặc định (Built-in Control Flow) tích hợp sẵn trong trình biên dịch của Angular.

**So với ngFor/ngIf ngày xưa:** `ngFor` và `ngIf` là các Directive kiểu cũ, bắt buộc phải import `CommonModule` hoặc `NgFor/NgIf` vào mảng imports. Cú pháp mới `@for` / `@if` ngắn gọn hơn, chạy nhanh hơn và không cần import.

## PHẦN B – Tìm lỗi trong code

Mỗi đoạn có đúng một lỗi. Chỉ ra lỗi, giải thích tại sao sai, và viết lại cho đúng.

### Lỗi số 1

```
<button type="button" disabled="{{ soLy() === 1 }}">-</button>
```

**Lỗi ở đâu:** `disabled="{{ soLy() === 1 }}"`

**Tại sao sai:** Gán thuộc tính điều kiện (boolean property) bằng interpolation `{{ }}` sẽ biến giá trị thành chuỗi `"true"/"false"`, làm nút bấm luôn bị disabled.

**Sửa lại:**

```
<button type="button" [disabled]="soLy() === 1">-</button>
```

### Lỗi số 2

```
// File: src/app/drink-detail/drink-detail.ts
import { Component, input } from '@angular/core';
import { DrinkModel } from '../models';

@Component({
  selector: 'app-drink-detail',
  templateUrl: './drink-detail.html',
  styleUrls: ['./drink-detail.css'],
})
export class DrinkDetail {
  readonly drink = input.required<DrinkModel>();
}
```

**Lỗi ở đâu:** `import { DrinkModel } from '../models';`

**Tại sao sai:** File `drink-detail.ts` nằm trong thư mục con `src/app/drink-detail/`. Muốn lấy file `models.ts` ở ngoài `src/app/` phải dùng `'../models'`.

**Sửa lại:**

```
import { DrinkModel } from '../../models';
```

### Lỗi số 3

```
// File: src/app/drink-list/drink-list.ts
import { Component } from '@angular/core';
import { DrinkDetail } from '../drink-detail/drink-detail';
```

```

@Component({
  selector: 'app-drink-list',
  imports: [],
  templateUrl: './drink-list.html',
  styleUrls: ['./drink-list.css'],
})
export class DrinkList {}

<!-- drink-list.html -->
<app-drink-detail [drink]="selectedDrink()" />

```

**Lỗi ở đâu:** imports: [] trong DrinkList

**Tại sao sai:** File drink-list.html gọi thẻ <app-drink-detail /> nhưng trong mảng imports của DrinkList lại chưa đăng ký DrinkDetail.

**Sửa lại:**

imports: [DrinkDetail],

#### Lỗi số 4

```

<app-drink-detail [drink]="selectedDrink" />

```

**Lỗi ở đâu:** [drink]="selectedDrink"

**Tại sao sai:** selectedDrink là một Signal. Thiếu cặp ngoặc () nên bạn đang truyền cả hàm Signal thay vì giá trị bên trong.

**Sửa lại:**

```

<app-drink-detail [drink]="selectedDrink()" />

```

#### Lỗi số 5

```

export class DrinkDetail {
  protected readonly drink = input.required<DrinkModel>();
}

```

**Lỗi ở đâu:** protected readonly drink = input.required<DrinkModel>();

**Tại sao sai:** input() là cổng nhận dữ liệu do component cha truyền vào, không được để từ khóa protected.

**Sửa lại:**

```

export class DrinkDetail {
  readonly drink = input.required<DrinkModel>();
}

```

#### Lỗi số 6

```

<ul>
  @for (topping of toppingCanDung()) {
    <li>{{ topping.name }}</li>
  }
</ul>

```

**Lỗi ở đâu:** Cú pháp @for thiếu thuộc tính track.

**Tại sao sai:** Trong Angular 17+, cú pháp @for bắt buộc phải có track để xác định danh tính từng phần tử trong danh sách.

**Sửa lại:**

```
<ul>
  @for (topping of toppingCanDung(); track topping.name) {
    <li>{{ topping.name }}</li>
  }
</ul>
```

## Lỗi số 7

```
import { Component, signal } from '@angular/core';
import { NgIf } from '@angular/common';

@Component({
  selector: 'app-drink-list',
  imports: [NgIf],
  templateUrl: './drink-list.html',
  styleUrls: ['./drink-list.css'],
})
export class DrinkList {
  protected readonly drinks = signal(MOCK_DRINKS);
}

@if (drink.isPopular) {
  <span>★</span>
}
```

**Lỗi ở đâu:** imports: [NgIf] và biến drink không tồn tại trong class.

**Tại sao sai:** @if là cú pháp mới tích hợp sẵn, không cần import NgIf. Ngoài ra biến drink chưa được định nghĩa trong class (chỉ có signal drinks).

**Sửa lại:**

```
TypeScript
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-drink-list',
  imports: [], // Không cần import NgIf
  templateUrl: './drink-list.html',
  styleUrls: ['./drink-list.css'],
})
export class DrinkList {
  protected readonly drink = signal(MOCK_DRINKS[0]); // Sửa lại thành 1 món cụ thể
```

```

    }
    HTML
    @if (drink().isPopular) {
        <span>★</span>
    }

```

### Lỗi số 8

```

@for (drink of drinks(); track drink.id) {
    <button [class.is-active]="selectedDrink().id drink.id">{{ drink.name }}</button>
}

```

**Lỗi ở đâu:** [class.is-active]="selectedDrink().id drink.id"

**Tại sao sai:** Thiếu toán tử so sánh === giữa hai id và drink.id thiếu cặp ngoặc nếu nó là signal, hoặc trong vòng lặp @for (drink of drinks()) thì drink là một item object thực sự. Dấu so sánh bị thiếu/viết sai.

**Sửa lại:**

```

@for (drink of drinks(); track drink.id) {
    <button [class.is-active]="selectedDrink().id === drink.id">
        {{ drink.name }}
    </button>
}

```

### Lỗi số 9

```

@for (drink of drinks; track drink.id) {
    <li>{{ drink.name }}</li>
}

```

**Lỗi ở đâu:** @for (drink of drinks; track drink.id)

**Tại sao sai:** drinks trong class là một Signal (drinks = signal(...)). Khi duyệt mảng trong @for, bắt buộc phải gọi ngoặc drinks() để lấy mảng dữ liệu thực sự.

**Sửa lại:**

```

@for (drink of drinks(); track drink.id) {

    <li>{{ drink.name }}</li>

}

```